

II - TP 2

Syntaxe de base de Python

1 Types et variables

1.1 Opérations sur les types de base

On a vu que l'on peut écrire des instructions en Python dans la console, qui, lorsqu'elles sont exécutées, sont évaluées par l'interpréteur Python pour réaliser l'opération demandée. En particulier, on peut voir la console Python comme une grosse calculatrice : il est possible d'effectuer toutes les opérations mathématiques "usuelles".

Question 1. 1. Compléter le tableau ci-dessous à l'aide de la console Python de Thonny.

Description mathématique	Expression python	Résultat
Addition •	• <code>8 * 3</code>	=
Soustraction •	• <code>34 % 4</code>	=
Multiplication •	• <code>9 - 100</code>	=
Division •	• <code>2 + 7</code>	=
Mise à la puissance •	• <code>5 / 2</code>	=
Division euclidienne : quotient •	• <code>20 // 6</code>	=
Division euclidienne : reste •	• <code>2**4</code>	=

2. Attention. En Python, comment note-t-on le nombre décimal 0,25 ?

Les calculatrices possèdent souvent des fonctions **prédéfinies**, comme sinus, cosinus, tangente, ou encore la fonction racine carrée. On peut également utiliser des **constantes usuelles**, comme le nombre π .

Question 2. Dans la console, évaluer l'expression `cos(60)`. Que constate-t-on ?

Dans tout langage informatique, on utilise des **bibliothèques** de fonctions. Il s'agit de collections de fonctions prédéfinies, utilisables directement par le programmeur. Celles-ci sont regroupées par "catégorie" : par exemple une bibliothèque pour les fonctions mathématiques de base. Nous avons déjà utilisé une bibliothèque dans le TP précédent : la bibliothèque `turtle` met à disposition des fonctions qui permettent d'interagir graphiquement avec une tortue à l'écran.

Question 3. 1. Exécuter les instructions suivantes. Que constate-t-on ?

Code python

```

1 from math import cos
2 cos(60)
```

2. La bibliothèque Python `math` contient également les fonctions `sin`, `tan`, ainsi que la fonction `sqrt` (pour "square root", racine carrée en anglais).

À l'aide de la console Thonny, calculer $\sqrt{5}$. Arrondir au centième.

3. Exécuter les instructions ci-dessous. Que constate-t-on ?

Code python

```

1 import math
2 math.sqrt(16)
3 help(math)
```

On a vu que l'on peut effectuer des opérations arithmétiques sur nombres entiers ou *flottants* avec Python. Il est également possible de les **comparer**.

Question 4. Évaluer les expressions suivantes à l'aide de l'interpréteur Python.

1. $2 + 2 == 4$

3. $2 * 8 != 10$

5. $(5 + 3) * 2 <= 10$

2. $3 - 1 == 5$

4. $2 ** 3 != 8$

6. $5 + 3 * 2 > 10$

Les comparaisons précédentes renvoient des **valeurs** spéciales : True si la comparaison est vraie, False sinon. Ces deux valeurs forment l'ensemble des **booléens**. Il est également possible d'effectuer des opérations logiques sur les booléens.

Question 5. 1. Évaluer les opérations suivantes à l'aide de l'interpréteur Python.

a. True and False

c. True or False

e. not (False or True)

b. True and True

d. not True

f. not False or True

2. Si a et b sont deux booléens, expliquer par une phrase en français quand les expressions suivantes valent True :

a. a and b

b. a or b

c. not a

3. Si a et b sont deux booléens, expliquer quand est-ce que $a \wedge b$ vaut True.

Enfin, Python nous permet de manipuler du texte, que l'on appelle dans le vocabulaire informatique des **chaînes de caractères**. On peut définir une chaîne de caractères de deux manières différentes : soit avec des guillemets doubles "Mon super texte", soit avec des guillemets simples 'Mon super texte'.

Question 6. 1. Dans l'éditeur de code, écrire et exécuter les instructions :

```
Code python
1 print("Hello World !")
2 print('Bonjour tout le monde !')
```

2. À l'aide de la fonction print, faire afficher le texte « Il paraît que "L'espoir fait vivre" ».

3. Si a et b sont deux chaînes de caractères, expliquer par une phrase en français en quoi s'évaluent les expressions suivantes :

a. a + b

b. a <= b

1.2 Variables et exécution pas à pas

Lorsque l'on souhaite écrire des programmes informatiques complexes, il est souvent utile de donner un nom aux résultats intermédiaires. On utilise pour cela le concept de **variable**, qui associe à un nom une valeur stockée dans la mémoire de l'ordinateur.

Question 7. 1. On considère les codes ci-dessous. Quels sont les noms de variables utilisés ?

```
Code python
1 a = 42
2 b = 24
3 c = 12
4 print(a, b, c)
```

```
Code python
1 a = 42
2 b = 24
3 a = 17
4 print(a, b)
```

```
Code python
1 a = 42
2 b = 24
3 a = a + b
4 print(a, b)
```

```
Code python
1 x = 7
2 x = x + 1
3 x = x * x
4 print(x)
```

2. Exécuter chacun des codes ci-dessus. Les résultats vous semblent-ils surprenants ?

Afin de mieux comprendre le déroulement d'un programme informatique, on utilise le mode **déboguer**, ou *exécution pas à pas*. Pour entrer en mode déboguer :

- Dans le menu "Affichage", cliquer sur l'item "Variables".
- Démarrer le mode débogueur en cliquant sur l'icône en forme d'insecte 🐛.
- Exécuter pas à pas le programme en cliquant sur l'icône ⏸.

À l'aide du mode pas à pas, on peut dresser une **trace d'exécution** du programme : il s'agit de l'évolution, ligne par ligne, des valeurs associées aux différentes variables présentes dans le code. On donne ci-contre la trace d'exécution du premier code de la question 7. Une case est noircie si la variable n'est pas encore affectée à ce moment de l'exécution du programme.

	a	b	c
Ligne 1	42		
Ligne 2	42	24	
Ligne 3	42	24	12
Ligne 4	42	24	12

Question 8. À l'aide du mode déboguer, donner les traces d'exécutions des différents codes présents dans la question 7.

2 Conditions et sauts

L'exécution d'un programme n'est pas toujours *linéaire*. Il arrive parfois que l'on "saute" à des instructions qui ne sont pas écrites sur la ligne suivante dans le code du programme. En général, ces "sauts" se font sous condition.

2.1 Exécution conditionnelle

En Python, il est possible d'utiliser le mot-clé `if` pour exécuter un bloc d'instructions (défini par son indentation) si une certaine condition booléenne est vérifiée. On peut éventuellement utiliser le mot-clé `else` pour spécifier un bloc de code à exécuter si la condition n'est pas vérifiée. On considère les codes ci-dessous.

Code python

```

1 if 12 * 2 == 24:
2     print("logique")
3 print("bien sûr")

```

Code python

```

1 if 12 * 2 == 42:
2     print("logique")
3 print("bien sûr")

```

Code python

```

1 if True:
2     print("Oui")
3 else:
4     print("Non")
5 print("Et alors")

```

Code python

```

1 if False:
2     print("Oui")
3 else:
4     print("Non")
5 print("Et alors")

```

Question 9. 1. Sans exécuter les codes, écrire sur votre cahier de TP quels sont les affichages réalisés dans chacun des cas.

2. Exécuter les codes Python pour vérifier votre réponse.

3. Indiquer le numéro des lignes qui **ne sont pas** exécutées par le programme ci-contre.

Code python

```

1 a = 3
2 b = 4
3 if a > b and a == 3:
4     print('vert')
5 else:
6     print('bleu')
7 if a == 3 or a < b:
8     print('jaune')

```

Il est possible d'utiliser le mot-clé `elif` (contraction de `else` et de `if`) si on veut préciser plusieurs alternatives possibles.

Question 10. On considère les deux codes suivants.

Code python

```
1 t = 40
2 if t > 100:
3     print("Évaporation")
4 elif t > 37.5:
5     print("Très chaud")
6 elif t > 20:
7     print("Agréable")
8 else:
9     print("Froid")
10 print("Fin de la météo")
```

Code python

```
1 t = 40
2 if t > 20:
3     print("Agréable")
4 elif t > 37.5:
5     print("Très chaud")
6 elif t > 100:
7     print("Évaporation")
8 else:
9     print("Froid")
10 print("Fin de la météo")
```

Recopier et exécuter les codes ci-dessus. Pourquoi les deux programmes n'affichent-ils pas la même chose ?

2.2 Répétition conditionnelle

En Python, on utilise le mot-clé `while` ("tant que" en anglais) pour répéter un bloc de code "tant que" une certaine condition booléenne est vraie. Tout le bloc indenté sous le `while` est exécuté. À la fin de l'exécution du bloc, on vérifie à nouveau si la condition est vraie. Si c'est le cas, on exécute à nouveau le bloc, sinon on exécute la prochaine ligne après le bloc.

Considérons le problème suivant : en 2024, Ahmed a versé sur son livret d'épargne 3000€. Chaque année, grâce aux intérêts perçus, la somme disponible sur le livret est multipliée par 1,03.

On souhaite savoir après combien d'années la somme disponible sur le livret d'Ahmed sera supérieure à 10 000€. Pour cela, on a écrit le code Python ci-contre.

Code python

```
1 s = ...
2 a = 2024
3 while s < 10000:
4     s = ...
5     a = a + 1
6     print(a)
```

Question 11. 1. Recopier et modifier le code de l'énoncé. Celui-ci doit afficher la première année lors de laquelle Ahmed possédera plus de 10 000€.

2. En vous inspirant du code précédent, écrire un code Python qui affiche la somme totale possédée par Ahmed sur son compte ainsi que l'année en question pour chaque année jusqu'en 2040.

3 Retour sur les boucles

Lorsque l'on connaît à l'avance le nombre maximum de fois que l'on doit répéter un bloc d'instructions, on utilise le mot-clé `for`, ainsi que la fonction `range`. Dans tous les codes ci-dessous, `i` est appelée la **variable de boucle**.

Pour mieux comprendre, on considère les codes suivants.

Code python

```
1 for i in range(3):
2     print("Bonjour")
```

Code python

```
1 for i in range(3, 10):
2     print(i)
```

Code python

```
1 for i in range(3):
2     print(i)
```

Code python

```
1 for i in range(3, 10, 2):
2     print(i)
```

Question 12. 1. Recopier et exécuter les codes ci-dessus.

2. Associer chacun des codes ci-dessus à sa description correcte en français.

- Pour i allant de 3 à 9 de 2 en 2
- Pour i allant de 3 à 9
- Pour i allant de 0 (inclus) à 3 (exclu)
- Pour i allant de 0 à 2

Il faut **bien comprendre** que la boucle **for** **affecte** à la variable de boucles différentes valeurs lors des exécutions successives du bloc à répéter.

Question 13. 1. **Sans utiliser Thonny**, compléter le tableau et en déduire la valeur affichée par le programme.

Code python

```
1 S = 0
2 for i in range(1, 6):
3     S = S + i
4 print(S)
```

	i	S
Avant le début de la boucle		
Fin de la 1 ^{re} itération	1	
Fin de la 2 ^e itération	2	
Fin de la 3 ^e itération		
Fin de la 4 ^e itération		
Fin de la 5 ^e itération		

2. Vérifier votre réponse en exécutant le code Python.

3. Écrire uniquement avec des additions le calcul réalisé par cet algorithme : $S = \dots$

4 Retour sur les fonctions

On a vu lors du TP d'introduction que les fonctions permettent de réutiliser des morceaux de code. Afin de faciliter la compréhension de la tâche réalisée par une fonction, on les accompagne *systématiquement* de leur documentation :

- la **signature** de la fonction : le type du ou des arguments d'appel, le type de ou des valeurs renvoyées par la fonction ;
- une **description** de l'effet de la fonction.

Par exemple, la fonction `decoupe` prend en argument un flottant `x` et renvoie un entier et un flottant, correspondant respectivement à la partie entière et à la partie fractionnaire du nombre `x`.

Code python

```
1 def decoupe(x):
2     """ float -> int, float
3     Renvoie la partie entière et fractionnaire de x """
4     partie_entiere = int(x)
5     partie_fractionnaire = x - int(x)
6     return partie_entiere, partie_fractionnaire
```

Code python

```
1 temp = 30.25
2 a, b = decoupe(temp)
3 print(a)
4 print(b)
```

⚙️ ➡️ Résultat

30
0.25

- Question 14.**
1. a. Écrire une fonction `fois2` qui prend en argument un entier `a` et qui renvoie le double de cet entier.
 - b. Écrire le code Python correspondant aux opérations suivantes.
Affecter à la variable `var1` l'entier 42. Appeler la fonction `fois2` avec comme argument `var1`, affecter le résultat à la variable `res`. Afficher `res`.
 2. a. Écrire une fonction `perimetre_aire` qui prend en argument un flottant `r` et qui renvoie le périmètre et l'aire du cercle de rayon `r`.
 - b. En faisant appel à la fonction `perimetre_aire`, écrire un code Python qui affiche l'aire d'un cercle de rayon 2.

Lorsqu'une instruction contenant le mot-clé `return` est exécutée dans le corps d'une fonction, cela a pour effet de terminer l'appel courant de la fonction et de renvoyer la valeur correspondante immédiatement.

```

1  def les_petits_passent(n):
2      """ int -> bool """
3      if n > 10:
4          return False
5      print("Petit !")
6      return True

```

```

Code python
a = les_petits_passent(3) # True
b = les_petits_passent(13) # False

```

⚙️ ➤ Résultat

Petit !

Question 15. On considère les fonctions `trop_long` et `trop_court` suivantes, qui prennent toutes les deux un entier `seuil` en argument.

```

Code python
1  def trop_long(seuil):
2      for i in range(100000):
3          if i > seuil:
4              return False
5          print(i)
6      return True

```

```

Code python
1  def trop_court(seuil):
2      for i in range(100000):
3          if i > seuil:
4              return False
5          print(i)
6      return True

```

1. Quel est le type de la valeur de retour de ces fonctions ?
2. Sans exécuter ces codes, déterminer l'affichage réalisé lorsque l'on exécute `trop_long(5)`.
3. Sans exécuter ces codes, déterminer l'affichage réalisé lorsque l'on exécute `trop_court(5)`.
4. Vérifier vos réponses en exécutant les codes.

Question 16. On considère le code des deux programmes ci-dessous.

```

Code python
1  def f(a):
2      a = a*2
3      return a
4
5  print(f(5))
6  print(a)
7  print(f(a))

```

```

Code python
a = 100

1  def f(a):
2      a = a*2
3      return a
4
5  print(f(5))
6  print(a)
7  print(f(a))
8
9

```

1. Exécuter ces programmes.
2. Cela est-ce surprenant ? Expliquer le comportement observé, en vous appuyant sur l'outil d'exécution étape par étape.